

Document number: P0342R0

Date: 2016-05-30

Reply-To: Mike Spertus, Symantec (mike_spertus@symantec.com)

Audience: Evolution Working Group

Timing barriers

Abstract

Timing blocks of code for profiling purposes and to compare different code approaches is central to production-quality software development. However, modern compiler reordering of code invalidates such measurements. This paper proposes a simple implementation-defined timing fence to support such use cases.

The problem

When teaching my students C++, I wanted to demonstrate how poor the performance of the naive implementation of the Fibonacci sequence is and ran the following code.

```
#include<chrono>
#include<atomic>
#include<iostream>
using namespace std;

size_t
fib(size_t n)
{
    if (n == 0)
        return n;
    if (n == 1)
        return 1;
    return fib(n - 1) + fib(n - 2);
}

int const which{42};

int main()
{
    auto start = chrono::high_resolution_clock::now();
    auto result = fib(which);
    auto end = chrono::high_resolution_clock::now();
    cout << "fib(" << which << ") is " << result << endl;
    cout << "Elapsed time is " << chrono::duration_cast(end - start).count() << "ms" << endl;

    return 0;
}
```

I was surprised to discover that the program printed an elapsed time of zero milliseconds and quickly realized that I had embarrassingly neglected the possibility that the compiler would reorder the calculation out of the timing region. However, when I tried to fix this problem to get valid timings, I realized that the *as-if rule* means that there is no standards-compliant way to do this basic activity. It was tempting to try using mutexes or atomic fences, which also inhibit code motion, but only as regards multiple threads and therefore impose no requirements on this single threaded program. I was able to invoke separate compilation to get a proper demonstration, but even that is vulnerable to whole program optimizations and link-time code generation, etc. As non-local compiler optimizations improve, the ability to benchmark algorithms in a portable way will become increasingly degraded.

The proposal

Without decimating the *as-if rule*, there appears to be no way to normatively require such timings to be correct. Nevertheless, timing a block of code or an algorithm is not devoid of meaning. This suggests the following simple proposal: Add an implementation-defined `std::timing_fence()` function that is a hint that code should not be moved from one side of the fence to the other. While compiler writers are free to ignore it as a matter of compliance, better capturing of the code into the benchmarked region would definitely qualify as a real-world improved quality-of-implementation.